

Menu-Driven Program for Square Matrix Diagonal Operations

Submitted By

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a menu-driven C program to read a variable-length square integer array and perform the following operations using separate functions:

1. Determine whether all the elements on its main diagonal are distinct.
2. Find the sum of its main diagonal elements.
3. Display its elements using a diagonal traversal. Begin with the element in the top-left corner and traverse successive diagonals alternately in the upward-right and downward-left directions.
4. Exit.

Example

For the input matrix with rows (1 2 3), (4 5 6) and (7 8 9):

- Main diagonal elements: 1 5 9. All the main diagonal elements are distinct.
- Sum of main diagonal elements: 15.
- Diagonal traversal: 1 2 4 7 5 3 6 8 9.

2. Objective and Concepts Used

The objective of this project is to build a modular, menu-driven C program that works on a square matrix whose size is decided by the user at run time. The following programming concepts of the first semester are applied:

- Two-dimensional arrays (matrix) and nested loops
- Dynamic memory allocation using malloc() and free() for a variable-length matrix
- User-defined functions, function prototypes and passing pointers to functions
- The switch-case statement inside a do-while loop for the menu
- Basic input validation using the return value of scanf()

3. Approach and Algorithm

3.1 Function Overview

Function	Purpose
createMatrix()	Allocates memory for an $n \times n$ matrix dynamically.
freeMatrix()	Releases the memory allocated to the matrix.
readMatrix()	Reads the matrix elements row by row.
displayMatrix()	Prints the matrix in tabular form.
checkDistinctDiagonal()	Operation 1: checks if all main diagonal elements are distinct.
sumMainDiagonal()	Operation 2: finds the sum of main diagonal elements.
diagonalTraversal()	Operation 3: prints the zig-zag diagonal traversal.

3.2 Operation 1: Distinct Diagonal Elements

The main diagonal consists of the elements $a[i][i]$ for $i = 0$ to $n-1$. Each diagonal element is compared with every element that comes after it. If any two are equal, the elements are not distinct.

1. Set flag distinct = 1.
2. For every i from 0 to $n-2$, and for every j from $i+1$ to $n-1$, compare $a[i][i]$ with $a[j][j]$.
3. If they are equal, set distinct = 0 and stop checking.
4. If distinct is still 1, print "distinct"; otherwise print "NOT distinct".

3.3 Operation 2: Sum of Main Diagonal

1. Set sum = 0.
2. For i from 0 to $n-1$, add $a[i][i]$ to sum.
3. Display sum.

3.4 Operation 3: Diagonal (Zig-Zag) Traversal

All elements $a[r][c]$ lying on the same diagonal have the same value of $r + c$. Let this value be d . The value of d ranges from 0 to $2n-2$, giving $2n-1$ diagonals in total. The direction alternates with d :

- If d is even, traverse upward-right (row decreases from the bottom to the top of that diagonal).
- If d is odd, traverse downward-left (row increases from the top to the bottom of that diagonal).

For a diagonal d , the valid rows are from low = 0 (if $d < n$, otherwise $d-n+1$) to high = d (if $d < n$, otherwise $n-1$). The column is always $c = d - r$.

3.5 Dry Run for the Example Matrix

d	Direction	Elements (row, col)	Output
0	-	(0,0)	1
1	Downward-left	(0,1), (1,0)	2 4
2	Upward-right	(2,0), (1,1), (0,2)	7 5 3
3	Downward-left	(1,2), (2,1)	6 8
4	Upward-right	(2,2)	9

Final traversal: **1 2 4 7 5 3 6 8 9**

4. Source Code

```
/*
 * Program : Menu-driven program on a square matrix
 * Author  : Rudra Pratap Singh (SAP ID: 590032811, Batch 16)
 *
 * Operations:
 * 1. Check whether all main-diagonal elements are distinct
 * 2. Find the sum of the main-diagonal elements
 * 3. Display the matrix using a zig-zag diagonal traversal
 * 4. Exit
 */

#include <stdio.h>
#include <stdlib.h>

/* Function prototypes */
int  **createMatrix(int n);
void  freeMatrix(int **a, int n);
void  readMatrix(int **a, int n);
void  displayMatrix(int **a, int n);
void  checkDistinctDiagonal(int **a, int n);
void  sumMainDiagonal(int **a, int n);
void  diagonalTraversal(int **a, int n);

int main(void)
{
    int n, choice;
    int **a;

    printf("=====\n");
    printf("    SQUARE MATRIX OPERATIONS (MENU)\n");
    printf("=====\n");

    /* Read the (variable) order of the square matrix */
    printf("Enter the order of the square matrix (n): ");
    if (scanf("%d", &n) != 1 || n <= 0)
    {
        printf("Invalid order. Program terminated.\n");
        return 1;
    }

    a = createMatrix(n);
    if (a == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    readMatrix(a, n);
    printf("\nThe matrix entered is:\n");
    displayMatrix(a, n);

    do
    {
        printf("\n----- MENU ----- \n");
        printf("1. Check whether main diagonal elements are distinct\n");
        printf("2. Find the sum of main diagonal elements\n");
        printf("3. Display elements using diagonal (zig-zag) traversal\n");
        printf("4. Exit\n");
        printf("----- \n");
        printf("Enter your choice (1-4): ");

        if (scanf("%d", &choice) != 1)
        {
            printf("Invalid input! Please enter a number.\n");
            while (getchar() != '\n')
                ; /* clear the input buffer */
        }
    }
}
```

```

        choice = 0;
        continue;
    }

    switch (choice)
    {
        case 1:
            checkDistinctDiagonal(a, n);
            break;
        case 2:
            sumMainDiagonal(a, n);
            break;
        case 3:
            diagonalTraversal(a, n);
            break;
        case 4:
            printf("Exiting the program. Thank you!\n");
            break;
        default:
            printf("Invalid choice! Please enter 1, 2, 3 or 4.\n");
    }
    } while (choice != 4);

    freeMatrix(a, n);
    return 0;
}

/* Allocates memory for an n x n matrix */
int **createMatrix(int n)
{
    int i;
    int **a = (int **)malloc(n * sizeof(int *));
    if (a == NULL)
        return NULL;

    for (i = 0; i < n; i++)
    {
        a[i] = (int *)malloc(n * sizeof(int));
        if (a[i] == NULL)
            return NULL;
    }
    return a;
}

/* Releases the memory used by the matrix */
void freeMatrix(int **a, int n)
{
    int i;
    for (i = 0; i < n; i++)
        free(a[i]);
    free(a);
}

/* Reads the elements of the matrix */
void readMatrix(int **a, int n)
{
    int i, j;
    printf("Enter the %d elements of the matrix (row-wise):\n", n * n);
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
}

/* Displays the matrix in tabular form */
void displayMatrix(int **a, int n)
{
    int i, j;
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)

```

```

        printf("%5d", a[i][j]);
        printf("\n");
    }
}

/* Operation 1: checks whether all main diagonal elements are distinct */
void checkDistinctDiagonal(int **a, int n)
{
    int i, j, distinct = 1;

    printf("Main diagonal elements: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i][i]);

    for (i = 0; i < n - 1 && distinct; i++)
        for (j = i + 1; j < n; j++)
            if (a[i][i] == a[j][j])
            {
                distinct = 0;
                break;
            }

    if (distinct)
        printf("\nAll the main diagonal elements are distinct.\n");
    else
        printf("\nAll the main diagonal elements are NOT distinct.\n");
}

/* Operation 2: finds the sum of the main diagonal elements */
void sumMainDiagonal(int **a, int n)
{
    int i, sum = 0;
    for (i = 0; i < n; i++)
        sum += a[i][i];
    printf("Sum of main diagonal elements: %d\n", sum);
}

/* Operation 3: zig-zag diagonal traversal starting from the top-left.
 * Elements with the same (row + col) value lie on the same diagonal d.
 * Odd d -> traverse downward-left (row increases)
 * Even d -> traverse upward-right (row decreases) */
void diagonalTraversal(int **a, int n)
{
    int d, r, low, high;

    printf("Diagonal traversal: ");
    for (d = 0; d <= 2 * n - 2; d++)
    {
        low = (d < n) ? 0 : d - n + 1; /* smallest valid row */
        high = (d < n) ? d : n - 1; /* largest valid row */

        if (d % 2 == 1)
            for (r = low; r <= high; r++)
                printf("%d ", a[r][d - r]);
        else
            for (r = high; r >= low; r--)
                printf("%d ", a[r][d - r]);
    }
    printf("\n");
}

```

5. Sample Input and Output

Note: the menu is displayed in full before every choice while the program runs; it is shown only once in each test case below to save space.

Test Case 1: 3 x 3 matrix (example from the problem)

```
=====
      SQUARE MATRIX OPERATIONS (MENU)
=====
Enter the order of the square matrix (n): 3
Enter the 9 elements of the matrix (row-wise):
1 2 3
4 5 6
7 8 9

The matrix entered is:
  1   2   3
  4   5   6
  7   8   9

----- MENU -----
1. Check whether main diagonal elements are distinct
2. Find the sum of main diagonal elements
3. Display elements using diagonal (zig-zag) traversal
4. Exit
-----
Enter your choice (1-4): 1
Main diagonal elements: 1 5 9
All the main diagonal elements are distinct.

Enter your choice (1-4): 2
Sum of main diagonal elements: 15

Enter your choice (1-4): 3
Diagonal traversal: 1 2 4 7 5 3 6 8 9

Enter your choice (1-4): 4
Exiting the program. Thank you!
```

Test Case 2: 3 x 3 matrix with repeated diagonal elements

```
=====
SQUARE MATRIX OPERATIONS (MENU)
=====
Enter the order of the square matrix (n): 3
Enter the 9 elements of the matrix (row-wise):
5 2 3
4 5 6
7 8 5

The matrix entered is:
    5    2    3
    4    5    6
    7    8    5

----- MENU -----
1. Check whether main diagonal elements are distinct
2. Find the sum of main diagonal elements
3. Display elements using diagonal (zig-zag) traversal
4. Exit
-----
Enter your choice (1-4): 1
Main diagonal elements: 5 5 5
All the main diagonal elements are NOT distinct.

Enter your choice (1-4): 2
Sum of main diagonal elements: 15

Enter your choice (1-4): 3
Diagonal traversal: 5 2 4 7 5 3 6 8 5

Enter your choice (1-4): 7
Invalid choice! Please enter 1, 2, 3 or 4.

Enter your choice (1-4): 4
Exiting the program. Thank you!
```

Test Case 3: 4 x 4 matrix

```
=====
SQUARE MATRIX OPERATIONS (MENU)
=====
Enter the order of the square matrix (n): 4
Enter the 16 elements of the matrix (row-wise):
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

The matrix entered is:
  1   2   3   4
  5   6   7   8
  9  10  11  12
 13  14  15  16

----- MENU -----
1. Check whether main diagonal elements are distinct
2. Find the sum of main diagonal elements
3. Display elements using diagonal (zig-zag) traversal
4. Exit
-----
Enter your choice (1-4): 1
Main diagonal elements: 1 6 11 16
All the main diagonal elements are distinct.

Enter your choice (1-4): 2
Sum of main diagonal elements: 34

Enter your choice (1-4): 3
Diagonal traversal: 1 2 5 9 6 3 4 7 10 13 14 11 8 12 15 16

Enter your choice (1-4): 4
Exiting the program. Thank you!
```

6. Conclusion

The program successfully reads a square matrix of any size entered at run time and performs the required operations through separate functions selected from a menu. The results for the sample matrix match the expected output given in the problem statement: the diagonal elements 1, 5 and 9 are distinct, their sum is 15, and the diagonal traversal is 1 2 4 7 5 3 6 8 9. The program was also tested with matrices having repeated diagonal elements and with a different order (4 x 4), and an invalid menu choice is handled with a proper message. This project helped in understanding arrays, dynamic memory allocation, functions and menu-driven programming in C.